BÁO CÁO THỰC HÀNH LAB 15

GIT FUNDAMENTALS

Người báo cáo: Liamz07

1. Mục tiêu:

- Hiểu được vòng đời tệp tin trong hệ thống Git (Working Directory → Staging Area → Repository).
- Thành thạo việc khởi tạo và quản lý kho chứa cục bộ (Local Repository).
- Thực hiện lưu trữ phiên bản, theo dõi lịch sử thay đổi và truy xuất sự khác biệt của mã nguồn.
- Áp dụng chuẩn hóa các quy trình thao tác dòng lệnh (Command Line Interface).

2. Môi trường:

- **Hệ điều hành:** Ubuntu Linux / macOS / Windows (với Git Bash)
- **Công cụ:** Git CLI (Phiên bản 2.x trở lên).
- **Kiểm tra môi trường:** git –version

3. Nội dung thực hành và kết quả:

BÀI 1: KHỞI TẠO KHO CHỨA (GIT INIT)

- **Mục đích:** Tạo một môi trường quản lý mã nguồn độc lập cho dự án.
- **Thao tác thực thi:** mkdir my-git-project && cd my-git-project && git init.
- **Phân tích kết quả:** Git tự động khởi tạo thư mục ẩn .git bên trong my-git-project. Thư mục này chứa toàn bộ dữ liệu cấu hình, cây đối tượng (objects) và lịch sử commit của kho chứa.

BÀI 2: KIỂM TRA TRẠNG THÁI HỆ THỐNG (GIT STATUS)

- **Mục đích:** Theo dõi trạng thái hiện tại của các tệp tin trong working directory.
- **Thao tác thực thi:** git status.
- **Phân tích kết quả:** Terminal hiển thị thông tin nhánh mặc định (On branch main hoặc master) và báo cáo chưa có thay đổi nào (No commits yet).

BÀI 3: ĐƯA TỆP TIN VÀO STAGING AREA (GIT ADD)

- **Mục đích:** Đánh dấu các thay đổi chuẩn bị cho việc lưu trữ phiên bản (commit).
- **Thao tác thực thi:**

```
# Tạo tệp tin ban đầu
echo "Hello Git Fundamentals" > index.html

# Kiểm tra trạng thái (Hiển thị file Untracked)
git status

# Đưa file vào Staging Area
git add index.html

# Kiểm tra lại trạng thái (Hiển thị file Staged)
git status
```

- **Phân tích kết quả:** Tệp index.html chuyển từ trạng thái Untracked (màu đỏ) sang Changes to be committed (màu xanh), sẵn sàng đưa vào ảnh chụp phiên bản (snapshot).

BÀI 4: GHI NHẬN THAY ĐỔI VÀO REPOSITORY (GIT COMMIT)

- **Mục đích:** Lưu ảnh chụp cố định của project vào kho lịch sử kèm thông điệp mô tả.
- **Thao tác thực thi:** git commit -m "Initial commit: Add index.html"
- **Phân tích kết quả:** Git tạo ra một commit object mới mang mã băm SHA-1 duy nhất (ví dụ: a1b2c3d), lưu giữ toàn bộ trạng thái hiện tại của cây thư mục.

BÀI 5: KIỂM TRA SỰ KHÁC BIỆT NỘI DUNG (GIT DIFF)

- **Mục đích:** So sánh chi tiết thay đổi giữa Working Directory và Staging Area trước khi tạo commit mới.
- **Thao tác thực thi:**

```
# Sửa đổi nội dung tệp tin
echo "Add second line to file" >> index.html

# So sánh sự khác biệt
git diff
```

- **Phân tích kết quả:** Output hiển thị chi tiết dòng vừa được thêm vào với ký tự + Add second line to file, giúp lập trình viên rà soát mã nguồn chính xác.

BÀI 6: XEM LỊCH SỬ COMMIT (GIT LOG)

- **Mục đích:** Truy xuất danh sách và chi tiết các phiên bản đã lưu trong quá trình phát triển.
- **Thao tác thực thi:**

```
# Đưa thay đổi mới vào staging và commit
git add index.html
git commit -m "Update index.html with second line"

# Xem lịch sử chi tiết
git log

# Xem lịch sử rút gọn
git log --oneline
```

- **Phân tích kết quả:** Chuỗi lịch sử thể hiện thứ tự các commit theo mốc thời gian, tên tác giả, email và thông điệp commit tương ứng.

4. Tổng hợp các lệnh đã thực hành:

STT	Lệnh git	Cú pháp sử dụng	Ý nghĩa & Chức năng
1	git init	Git init	Khởi tạo kho chứa Git mới tại thư viện hiện hành.
2	git status	Git status	Kiểm tra trạng thái các tệp tin (Untracked, Modified, Staged).
3	git add	Git add <file> / git add .	Đưa tệp tin/tất cả thay đổi vào vùng Staging Area.
4	git commit	Git commit -m “message”	Ghi nhận các thay đổi từ Staging vào Repository.
5	git diff	Git diff	So sánh sự khác biệt giữa Working Directory và Staging Area.
6	git log	Git log / git log --oneline	Xem lịch sử các commit đã thực hiện.

5. Kết luận và bài học rút ra:

- **Kết quả đạt được:** Hoàn thành đầy đủ 6 thao tác nền tảng của Git, nắm vững luồng làm việc với 3 vùng không gian chính (Working Directory, Staging Area, Local Repository).
- **Kỹ năng tích lũy:** Biết cách kiểm soát các phiên bản mã nguồn, chủ động kiểm tra trạng thái và lịch sử thay đổi để tránh mất mát dữ liệu trong quá trình phát triển phần mềm.

HẾT!